



Comprehensive analysis of gradient-based hyperparameter optimization algorithms

O. Y. Bakhteev¹ · V. V. Strijov^{1,2}

Published online: 3 June 2019

© Springer Science+Business Media, LLC, part of Springer Nature 2019

Abstract

The paper investigates hyperparameter optimization problem. Hyperparameters are the parameters of model parameter distribution. The adequate choice of hyperparameter values prevents model overfit and allows it to obtain higher predictive performance. Neural network models with large amount of hyperparameters are analyzed. The hyperparameter optimization for models is computationally expensive. The paper proposes modifications of various gradient-based methods to simultaneously optimize many hyperparameters. The paper compares the experiment results with the random search. The main impact of the paper is hyperparameter optimization algorithms analysis for the models with high amount of parameters. To select precise and stable models the authors suggest to use two model selection criteria: cross-validation and evidence lower bound. The experiments show that the models optimized using the evidence lower bound give higher error rate than the models obtained using cross-validation. These models also show greater stability when data is noisy. The evidence lower bound usage is preferable when the model tends to overfit or when the cross-validation is computationally expensive. The algorithms are evaluated on regression and classification datasets.

Keywords Gradient descent · Hyperparameter optimization · Model selection · Neural networks · Classification · Regression

The research was made possible by Government of the Russian Federation (Agreement No. 05.Y09.21.0018) and FASIE Project No. 44116. This paper contains results of the project Statistical methods of machine learning, which is carried out within the framework of the Program “Center of Big Data Storage and Analysis” of the National Technology Initiative Competence Center. It is supported by the Ministry of Science and Higher Education of the Russian Federation according to the agreement between the M.V. Lomonosov Moscow State University and the Foundation of Project support of the National Technology Initiative from 11.12.2018, No 13/1251/2018.

✉ O. Y. Bakhteev
bakhteev@phystech.edu

¹ Moscow Institute of Physics and Technology, Dolgoprudny, Russia

² FRCCSC of the Russian Academy of Sciences, Moscow, Russia

1 Introduction

The paper analyzes hyperparameter optimization problem. *Model* is a function superposition, which solves a classification or regression problem. *Model hyperparameters* are the parameters of model parameter distribution.

The paper focuses on the neural network models. The parameter optimization problem is computationally expensive because of training loss non-convexity. The amount of model parameters can reach millions (Salakhutdinov and Hinton 2007) and such models optimization requires multiple days (Sutskever et al. 2014). The hyperparameter values can influence significantly the quality of models (Hwang and Jeong 2018; Nystrup et al. 2018). These two facts make the hyperparameter optimization problem very important. The adequate choice of hyperparameter values also helps to prune redundant parameters to make the model more compact and stable (Graves 2011; Karaletsos and Rätsch 2015). Comprehensive analysis of the hyperparameters and hyperparameter optimization algorithms leads to challenges addressed by operations research.

In this paper gradient-based algorithms are analyzed. They optimize a large amount of hyperparameters within a short time in contrast to gradient-free algorithms (Maclaurin et al. 2015). The complexity of hyperparameter optimization is comparable to the model parameter optimization complexity, therefore, the parameters and hyperparameters are optimized in a single procedure. As a baseline, a random search algorithm is used. The pros and cons of all the algorithms are illustrated in Table 1. Opposing to papers (Fu et al. 2016; Pedregosa 2016; Luketina et al. 2016) this research does not focus on the hold-out cross-validation and analyzes algorithms in general. The analyzed algorithms are implemented as a toolbox [2]. The authors propose a number of analyzed algorithms modifications: we extend all the algorithms to work with validation loss that can contain hyperparameters, for DrMad algorithm (Fu et al. 2016) we add an additional parameter that regularizes a length of analyzed parameter update trajectory, for HOAG algorithm (Pedregosa 2016) we use stochastic gradient descent for large linear system solving because of high dimension of the linear system. The main impact of the paper is the analysis of algorithm behavior with various validation loss functions and a quality and stability analysis of the resulting models. The experiments are conducted using cross-validation and evidence lower bound as a model selection criterion. Opposing to the paper (Pedregosa 2016), where the optimization algorithms comparison can also be found, this paper presents the results on the large datasets, such as WISDM (Kwapisz et al. 2011) and MNIST (LeCun 1998), where the number of hyperparameters is significant. The experimental results show that the gradient-based algorithms are significantly more effective than random search-based algorithms when the number of hyperparameters is large.

Related works The papers (Bergstra and Bengio 2012; Bergstra et al. 2011) proposed hyperparameters optimization strategies based on the random search. The current gold-standard methods (Snoek et al. 2015; Hutter et al. 2011) proposed hyperparameter optimizations based on the probabilistic models construction for the optimal hyperparameter value prediction. These methods are ineffective whenever the number of hyperparameters is large (Maclaurin et al. 2015).

The gradient-based hyperparameter optimization methods were suggested in Maclaurin et al. (2015), Domke (2012), Fu et al. (2016), Pedregosa (2016), Luketina et al. (2016). The papers (Fu et al. 2016; Maclaurin et al. 2015) proposed the gradient-based methods that use reverse differentiation method. This method is similar to backpropagation. The main idea of this approach is to restore the full history of parameter updates in order to optimize hyper-

Table 1 The analyzed algorithms properties

Algorithm	Type	Pros	Cons
Random search	Stochastic	Easy to implement	Ineffective in high dimensions (curse of dimension)
Greedy (Luketina et al. 2016)	Gradient-based	Can be used in inner model parameter optimization	Greedy and non-optimal: the algorithm is correct whenever the loss function Hessian is identical.
HOAG (Pedregosa 2016)	Gradient-based	Fast convergence	The algorithm requires additional parameter configuration: it is required to solve the linear system, which depends on the Hessian of a loss function. The error in linear system solution influences the final model quality.
DrMAD (Fu et al. 2016)	Gradient-based	Considers the trajectory of parameter updates and the optimization algorithm.	Can be instable because of gradient explosion or vanishing. The algorithm is correct when the trajectory of parameter update is linear, which is mostly not true for the complex non-convex models.

parameters using this history. In general this procedure is computationally expensive. The authors proposed to use only the last update of the parameters at each optimization iteration in Luketina et al. (2016). This method can be considered as a greedy hyperparameter optimization algorithm. The authors optimized the model parameters using stochastic gradient descent with momentum in Maclaurin et al. (2015). This allowed to use less memory for storing the parameter update history. The authors of the paper (Fu et al. 2016) proposed to linearize the history trajectory to effectively restore the update history. The authors of the paper (Pedregosa 2016) used an approximation of the exact hyperparameter gradient, which can be calculated when the problem satisfies some regular conditions.

For the hyperparameter optimization various model selection criteria can be used (MacKay 2002; Bishop 2006). One of them is the marginal likelihood or *evidence* (MacKay 2002; Bishop 2006; Kuznetsov et al. 2016; Strijov 2014). The authors of the papers (Kuznetsov et al. 2016; Strijov 2014) considered the problem of model selection and hyperparameter optimization for the linear regression problem. One of the marginal likelihood approximation methods is an *evidence lower bound*, which can be obtained using variational inference (Bishop 2006). A stochastic version of the variational inference was proposed in Hoffman et al. (2013). The authors of the paper (Salimans et al. 2015) analyzed the relation between the gradient-based evidence lower bound estimations and MCMC methods. An alternative model selection criterion is a minimum description length (Grünwald 2005) that characterizes statistical complexity of the model. An evidence lower bound estimation method for the neural networks was proposed in Graves (2011). The authors analyzed the relation between the evidence lower bound and minimum description length.

The other model selection criterion is the cross-validation (Arlot and Celisse 2010; Kuznetsov et al. 2016). One of this criterion problem is its high computational cost (Vishnu et al. 2015; Krstajic et al. 2014). The authors of the paper (Bengio and Grandvalet 2004) analyzed the variance of cross-validation model performance estimation. In Kuznetsov et al. (2016) the authors compared hyperparameter values obtained during hyperparameter optimization using various model selection criteria.

2 Problem statement

There is given a dataset

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}, i = 1, \dots, m. \quad (1)$$

It contains the object matrix $[\mathbf{x}_1, \dots, \mathbf{x}_m]^T = \mathbf{X}$ and the label vector $[y_1, \dots, y_m]^T = \mathbf{y}$. The label y_i is in finite set, $y_i \in \mathbb{Y} = \{1, \dots, Z\}$, in case of Z -class classification problem. For the regression problem the label y_i is in the real-value subset $y_i \in \mathbb{Y} \subset \mathbb{R}$.

A model $\mathbf{f}(\mathbf{w}, \mathbf{x})$ predicts the variable y_i :

$$f : \mathbb{R}^u \times \mathbb{R}^n \rightarrow \mathbb{Y}, \quad \mathbf{w} \in \mathbb{R}^u.$$

It is differentiable with respect to parameters $\mathbf{w}$.

Introduce the probabilistic interpretation of the model $f(\mathbf{w}, \mathbf{x})$ for the classification and regression problems.

Regression problem Suppose the dependent variable y is normally distributed:

$$\mathbf{y} = \mathcal{N}(\mathbf{f}, \mathbf{I}), \quad \text{where } \mathbf{f} = \mathbf{f}(\mathbf{w}, \mathbf{X}) = [f(\mathbf{w}, \mathbf{x}_1), \dots, f(\mathbf{w}, \mathbf{x}_m)]^T. \quad (2)$$

Define the likelihood function $p(\mathbf{y}|\mathbf{X}, \mathbf{w})$:

$$\log p(\mathbf{y}|\mathbf{X}, \mathbf{w}) = -\frac{m}{2} \log(2\pi) - \frac{1}{2} (\mathbf{y} - \mathbf{f}(\mathbf{w}, \mathbf{X}))^T \mathbf{I} (\mathbf{y} - \mathbf{f}(\mathbf{w}, \mathbf{X})).$$

Classification problem For the 2-class classification problem suppose the dependent variable y is distributed binomially:

$$\mathbf{y} = \mathcal{B}(1 - \mathbf{f}, \mathbf{f}), \quad (3)$$

where $\mathbf{f}$ is the probability that objects from the matrix $\mathbf{X}$ belong to the first class. For the multiclass classification problem the dependent variable y is distributed multinomially and r -th component f_r is the probability that objects from the matrix $\mathbf{X}$ belong to the class r . Define the likelihood function

$$\log p(\mathbf{y}|\mathbf{X}, \mathbf{w}) = \sum_{\mathbf{x}, \mathbf{y} \in \mathbf{X}, \mathbf{y}} \sum_{r=1}^Z [y = r] \log f_r(\mathbf{w}, \mathbf{x}).$$

For both classification (3) and regression (2) problems define the prior distribution $p(\mathbf{w}|\mathbf{A})$:

$$\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{A}^{-1}), \quad (4)$$

where $\mathbf{A}^{-1} = \text{diag}[\alpha_1, \dots, \alpha_u]^{-1}$ is the covariance diagonal matrix. The hypotheses (2), (3) and (4) do not contradict each other since the normal distribution is unbounded (Farcomeni 2010).

In order to describe the general scheme of the model optimization procedure denote a vector of model parameters by $\mathbf{w}$. Denote by $\boldsymbol{\theta} \in \mathbb{R}^s$ a vector of parameters of the model parameters distribution. In the simple case $\boldsymbol{\theta}$ is a concatenation of vectors of parameters of multiple model instances. The details of $\boldsymbol{\theta}$ construction are given in the model selection methods description below.

Introduce an example of hyperparameter optimization using the coherent Bayesian inference. Optimize model parameters $\boldsymbol{\theta} = \mathbf{w}$ according to the model parameter distribution $p(\mathbf{w}|\mathbf{X}, \mathbf{y}, \mathbf{A})$:

$$\hat{\theta} = \arg \max(-L(\theta, \mathbf{A})) = p(\mathbf{w}|\mathbf{X}, \mathbf{y}, \mathbf{A}) = \frac{p(\mathbf{y}|\mathbf{X}, \mathbf{w})p(\mathbf{w}|\mathbf{A})}{p(\mathbf{y}|\mathbf{X}, \mathbf{A})}. \quad (5)$$

Calculate the hyperparameter $\mathbf{A}$ posterior distribution:

$$p(\mathbf{A}|\mathbf{X}, \mathbf{y}) \propto p(\mathbf{y}|\mathbf{X}, \mathbf{A})p(\mathbf{A}),$$

where $\propto$ means proportionality.

Suppose $p(\mathbf{A})$ is uniform on a large interval. The hyperparameter optimization problem is:

$$Q(\theta, \mathbf{A}) = p(\mathbf{y}|\mathbf{X}, \mathbf{A}) = \int_{\mathbf{w} \in \mathbb{R}^u} p(\mathbf{y}|\mathbf{X}, \mathbf{w})p(\mathbf{w}|\mathbf{A}) \rightarrow \max_{\text{diag}(\mathbf{A})=[\alpha_1, \dots, \alpha_u] \in \mathbb{R}^n}. \quad (6)$$

Formulate the hyperparameter optimization problem. Given functions L and Q that characterize the model selection method. The loss function L optimizes parameters θ . The validation function Q evaluates the model predictive performance. The problem is to optimize the parameters θ and the hyperparameters $\mathbf{A}$:

$$\hat{\mathbf{A}} = \arg \max_{[\alpha_1, \dots, \alpha_u] \in \mathbb{R}^n} Q(\hat{\theta}(\mathbf{A}), \mathbf{A}), \quad (7)$$

$$\hat{\theta}(\mathbf{A}) = \arg \min_{\theta \in \mathbb{R}^s} L(\theta, \mathbf{A}). \quad (8)$$

Introduce the functions L , Q and θ for various model selection methods.

Basic method Optimize the model parameters θ and hyperparameters $\mathbf{A}$ using the whole dataset $\mathcal{D}$ and the same function for both optimization and model evaluation with one model instance, $L = -Q$:

$$Q(\theta, \mathbf{A}) = -L(\theta, \mathbf{A}) = \log p(\mathbf{y}, \mathbf{w}|\mathbf{X}, \mathbf{A}) = \log p(\mathbf{y}|\mathbf{X}, \mathbf{w}) + \log p(\mathbf{w}|\mathbf{A}). \quad (9)$$

Describe two model selection methods that prevent model overfitting.

Cross-validation (Arlot and Celisse 2010). Split the dataset $\mathcal{D}$ into k equal parts: $\mathcal{D} = \mathcal{D}_1 \sqcup \dots \sqcup \mathcal{D}_k$.

Run k model optimizations for each subset. The vector θ is a concatenation of model parameters $\mathbf{w}_k$ for each optimization instance:

$$\theta = [\mathbf{w}_1, \dots, \mathbf{w}_k].$$

Construct the loss function L as an average negative log posterior probability over the remaining part of each split $\mathcal{D}_1, \dots, \mathcal{D}_k$:

$$L(\theta, \mathbf{A}) = -\frac{1}{k} \sum_{c=1}^k \left(\frac{k}{k-1} \log p(\mathbf{y} \setminus \mathbf{y}_c | \mathbf{X} \setminus \mathbf{X}_c, \mathbf{w}_c) + \log p(\mathbf{w}_c | \mathbf{A}) \right). \quad (10)$$

Construct the validation function Q as an average log likelihood over k splits:

$$Q(\theta, \mathbf{A}) = \frac{1}{k} \sum_{c=1}^k k \log p(\mathbf{y}_c | \mathbf{X}_c, \mathbf{w}_c). \quad (11)$$

Evidence Lower Bound (Graves 2011). As in the basic method description let $L = -Q$. It is an evidence lower bound:

$$\begin{aligned} \log p(\mathbf{y}|\mathbf{X}, \mathbf{A}) &\geq -D_{\text{KL}}(q(\mathbf{w})||p(\mathbf{w}|\mathbf{A})) + \int_{\mathbf{w}} q(\mathbf{w}) \log p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \mathbf{A}) d\mathbf{w} \approx \\ &\approx \sum_{i=1}^m \log p(y_i|\mathbf{x}_i, \mathbf{w}_i) - D_{\text{KL}}(q(\mathbf{w})||p(\mathbf{w}|\mathbf{A})) \\ &= -L(\boldsymbol{\theta}, \mathbf{A}) = Q(\boldsymbol{\theta}, \mathbf{A}), \end{aligned} \quad (12)$$

where q is an auxiliary distribution called variational distribution. Let q be a normal distribution:

$$q \sim \mathcal{N}(\boldsymbol{\mu}_q, \mathbf{A}_q^{-1}), \quad (13)$$

where $\mathbf{A}_q = \text{diag}[\alpha_1^q, \dots, \alpha_u^q]^{-1}$ is a diagonal covariance matrix and $\boldsymbol{\mu}_q$ is a mean vector. The divergence D_{KL} between 2 Gaussian variables is

$$D_{\text{KL}}(q(\mathbf{w})||p(\mathbf{w}|\mathbf{f})) = \frac{1}{2} (\text{Tr}[\mathbf{A}\mathbf{A}_q^{-1}] + (\boldsymbol{\mu} - \boldsymbol{\mu}_q)^\top \mathbf{A}(\boldsymbol{\mu} - \boldsymbol{\mu}_q) - u + \ln |\mathbf{A}^{-1}| - \ln |\mathbf{A}_q^{-1}|).$$

The vector of optimized parameters $\boldsymbol{\theta}$ is a vector of variational distribution q parameters:

$$\boldsymbol{\theta} = [\text{diag}(\mathbf{A}_q), \mu_1, \dots, \mu_u].$$

3 Gradient-based hyperparameter optimization methods

In this section we analyze hyperparameter optimization methods based on gradient descent. Suppose the parameters $\boldsymbol{\theta}$ are also optimized using gradient-based methods.

Definition 1 A stochastic gradient descent operator T estimates the parameters $\boldsymbol{\theta}'$ using their previous values $\boldsymbol{\theta}$ with random subset of dataset:

$$\boldsymbol{\theta}' = T(\boldsymbol{\theta}, \mathbf{A}, \mathcal{D}) = \boldsymbol{\theta} - \gamma \nabla L(\boldsymbol{\theta}, \mathbf{A})_{\mathcal{D}=\hat{\mathcal{D}}},$$

where $\hat{\mathcal{D}}$ is a random subset of $\mathcal{D}$.

Optimize the parameters $\boldsymbol{\theta}$ with η steps of stochastic gradient descent:

$$\hat{\boldsymbol{\theta}} = T \circ T \circ \dots \circ T(\boldsymbol{\theta}_0, \mathbf{A}) = T^\eta(\boldsymbol{\theta}_0, \mathbf{A}), \quad T(\boldsymbol{\theta}, \mathbf{A}) = \boldsymbol{\theta} - \gamma \nabla L(\boldsymbol{\theta}, \mathbf{A}), \quad (14)$$

where γ is the learning rate, $\boldsymbol{\theta}_0$ is the initial values of the vector $\boldsymbol{\theta}$.

Redefine the optimization problem according to the definition of operator T :

$$\hat{\mathbf{A}} = \arg \max_{\text{diag}(\mathbf{A}) \in \mathbb{R}^u} Q(T^\eta(\boldsymbol{\theta}_0, \mathbf{A})). \quad (15)$$

The general scheme of the hyperparameter optimization is the following:

1. In range from 1 to l , where l is the number of the hyperparameter optimization iterations;
2. Initialize parameters $\boldsymbol{\theta}$;
3. Solve the optimization problem (15) and obtain the new hyperparameter values $\hat{\mathbf{A}}$;
4. Set $\mathbf{A} = \hat{\mathbf{A}}$.

Table 2 The analyzed algorithms complexity and correctness

Algorithm	Type	Optimization iteration complexity	Correctness suppositions
Random search	Stochastic	$O(\eta s \hat{\mathcal{D}})$	–
Greedy (Luketina et al. 2016)	Gradient-based	$O(\eta u s \hat{\mathcal{D}})$	$\mathbf{H}(\theta) = \mathbf{I}$
HOAG (Pedregosa 2016)	Gradient-based	$O(\eta u s \hat{\mathcal{D}} + o)$, where o is a complexity of linear equation solution	First derivatives of Q and second derivatives of L are Lipschitz functions; $\det \mathbf{H} \neq 0$;
DrMAD (Fu et al. 2016)	Gradient-based	$O(\eta u s \hat{\mathcal{D}})$	Parameter trajectory $\theta = \theta_0, \dots, \theta_\eta$ is linear

The model is a multilayer perceptron optimized using backpropagation with the basic model selection method (9)

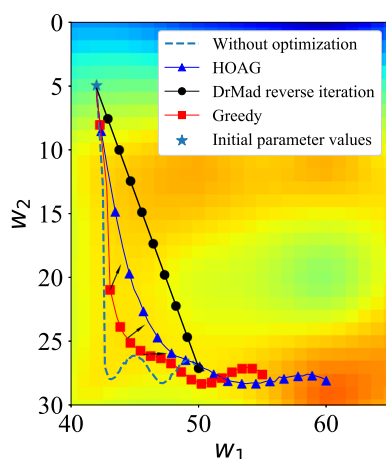


Fig. 1 An example of parameter update trajectories. The color displays the value of the validation function Q . Greedy algorithm optimizes hyperparameter during the parameter optimization, therefore it has the light blue trajectory between the optimized green trajectory of HOAG algorithm and the dark blue trajectory of parameters without hyperparameter optimization. DrMAD uses a linearized dashed parameter trajectory during hyperparameter optimization procedure. (Color figure online)

One of the methods for the problem (15) solution is to use a gradient descent not only for the parameters θ , but also for the hyperparameters $\mathbf{A}$. The exact gradient calculation $\nabla_{\mathbf{A}} Q(T^\eta(\theta_0, \mathbf{A}))$ is intractable because of the inner optimization operator T . Therefore the number of approximated solutions were presented. Their main properties are listed in Table 2. An example of the parameter and hyperparameter update with different hyperparameter optimization algorithms is illustrated in Fig. 1.

Greedy algorithm Update the hyperparameter $\mathbf{A}$ using gradient descent, which depends only on the last update of parameters θ . Optimize the hyperparameters and parameters in a single optimization procedure. Update the hyperparameter $\mathbf{A}$ at every iteration:

$$\hat{\mathbf{A}} = \mathbf{A} + \gamma_{\mathbf{A}} \nabla_{\mathbf{A}} Q(T(\theta, \mathbf{A}), \mathbf{A}) = \mathbf{A} + \gamma_{\mathbf{A}} \nabla_{\mathbf{A}} Q(\theta - \gamma \nabla L(\theta, \mathbf{A}), \mathbf{A}),$$

where $\gamma_{\mathbf{A}}$ is the learning rate for the hyperparameter optimization.

HOAG Obtain approximate hyperparameter gradient values $\nabla_{\mathbf{A}} Q(T^\eta(\theta_0, \mathbf{A}))$ using the following formula approximation of the of the gradient:

$$\nabla_{\mathbf{A}} Q(T^\eta(\theta_0, \mathbf{A})) = \nabla_{\mathbf{A}} Q(\theta, \mathbf{A}) - (\nabla_{\mathbf{A}} \nabla_{\theta} L(\theta, \mathbf{A}))^T \mathbf{H}(\theta)^{-1} \nabla_{\theta} Q(\theta, \mathbf{A}),$$

where $\mathbf{H}$ is the Hessian of the function L with respect to the parameters θ .

The algorithm of hyperparameter gradient $\nabla_{\mathbf{A}} Q$ approximation:

1. Optimize the parameters $\theta = T^\eta(\theta_0, \mathbf{A})$.
2. Solve linear system for a vector λ : $\mathbf{H}(\theta)\lambda = \nabla_{\theta} Q(\theta, \mathbf{A})$.
3. Compute the approximate hyperparameter gradient: $\hat{\nabla}_{\mathbf{A}} Q = \nabla_{\mathbf{A}} Q(\theta, \mathbf{A}) - \nabla_{\mathbf{A}} \nabla_{\theta} L(\theta, \mathbf{A})^T \lambda$.

The final update rule is

$$\hat{\mathbf{A}} = \mathbf{A} + \gamma_{\mathbf{A}} \hat{\nabla}_{\mathbf{A}} Q. \quad (16)$$

The computational cost of the Hessian $\mathbf{H}(\theta)$ evaluation from step 2 is high. In this paper we use stochastic gradient descent for solving this linear system.

DrMad For the hyperparameter gradient evaluation restore the full history of η parameters updates starting from the initial value θ_0 . For the simplification of this procedure suppose that the trajectory of parameters θ update is linear:

$$\theta^\tau = \theta_0 + \frac{\tau}{\eta} T(\theta). \quad (17)$$

The algorithm of approximate hyperparameter gradient calculation:

1. Optimize parameters $\theta = T^\eta(\theta_0, \mathbf{A})$.
2. Let $\hat{\nabla}_{\mathbf{A}} = \nabla_{\mathbf{A}} Q(\theta, \mathbf{A})$.
3. Let $\hat{\nabla}_{\theta} = \nabla_{\theta} Q(\theta, \mathbf{A})$.
4. In range from $\tau = \eta$ to 1:
5. Compute θ^τ (17).
6. $d\mathbf{v} = \gamma \hat{\nabla}_{\theta}$.
7. $\hat{\nabla}_{\mathbf{A}} = \hat{\nabla}_{\mathbf{A}} + d\mathbf{v} \nabla_{\mathbf{A}} \nabla_{\theta} L$.
8. $\hat{\nabla}_{\theta} = \hat{\nabla}_{\theta} - d\mathbf{v} \nabla_{\theta} \nabla_{\theta} L$.

The final update rule is analogous to (16). The algorithm is instable when the learning rate γ is large (Fu et al. 2016). For the stability of the algorithm we discard first 5% of values θ and calculate only each τ_{step} step of parameter updates history:

$$\theta^\tau = \theta_{\tau_0} + \frac{\tau}{\eta} T(\theta), \quad \tau \in \{\tau_0, \tau_0 + \tau_{\text{step}}, \dots, \eta - \tau_{\text{step}}, \eta\}, \quad (18)$$

where $\tau_0 = \lceil 0.05 \cdot \eta \rceil$.

4 Experiments

For the evaluation of the analyzed algorithms we conduct a series of computational experiments. A synthetic dataset [2], WISDM (Kwapisz et al. 2011) and MNIST (LeCun 1998) datasets are used. All datasets are split into Train and Test subsets. The algorithms are optimized on the Train subsets and evaluated on the Test subsets. For each dataset and each hyperparameter optimization algorithm we run the optimizations 5 times. The results are averaged. We use the following evaluation criteria.

1. Quality: the best value of Q : $\hat{Q} = \max_{j \in \{1, \dots, l\}} Q^j$.
2. Convergence: the number of iterations to have a validation value greater than 99% of the best value $\hat{Q}$:

$$\arg \min_j : \frac{Q^j - Q^0}{\hat{Q} - Q^0} \geq 0.99,$$

where Q^0 is the value of Q before the hyperparameter optimization.

3. Error function E :

$$E = \text{RMSE}(\mathbf{X}_{\text{test}}, \mathbf{y}_{\text{test}}) = \left(\frac{1}{|\mathbf{X}_{\text{test}}|} \sum_{\mathbf{x} \in \mathbf{X}_{\text{test}}, y \in \mathbf{y}_{\text{test}}} (f(\mathbf{x}, \mathbf{w}) - y)^2 \right)^{\frac{1}{2}}$$

for the regression problem and

$$E = \text{Acc}(\mathbf{X}_{\text{test}}, \mathbf{y}_{\text{test}}) = 1 - \frac{1}{|\mathbf{X}_{\text{test}}|} \sum_{\mathbf{x} \in \mathbf{X}_{\text{test}}, y \in \mathbf{y}_{\text{test}}} [f(\mathbf{x}, \mathbf{w}) \neq y]$$

for the classification problem.

4. The error function E_σ for the model with noisy dataset:

$$\begin{aligned} \text{RMSE}_\sigma &= \text{RMSE}(\mathbf{X}_{\text{test}} + \boldsymbol{\varepsilon}, \mathbf{y}_{\text{test}}), \\ \text{Acc}_\sigma &= \text{Acc}(\mathbf{X}_{\text{test}} + \boldsymbol{\varepsilon}, \mathbf{y}_{\text{test}}), \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma \mathbf{I}). \end{aligned}$$

The random search is used as a baseline. We set the number of the random search iterations to the number l of gradient-based hyperparameter optimization algorithms iterations. It is set to $l = 50$ for the synthetic dataset and WISDM dataset, $l = 25$ for the MNIST dataset. We use two types of loss function L and validation function Q :

1. Evidence lower bound for both L and Q (12);
2. Cross-validation functions (10),(11).

For the neural network models we set all the hyperparameters equal for each layer when use cross-validation (10). We use the full diagonal parameterization of the hyperparameters for all the linear models and for the neural network models when we optimize evidence lower bound (12).

The hyperparameters are initialized with uniform distribution $\mathcal{U}(a, b)$, where $a = -2$, $b = 10$ for the synthetic dataset and $a = -4$, $b = 10$ for the WISDM and MNIST datasets.

We calibrate the hyperparameter learning rate $\gamma_{\mathbf{A}}$ for each algorithm using grid search: $\{r \cdot 10^s, s \leq 1, r \in \{1, 25, 50, 75\}\}$. The largest value of $\gamma_{\mathbf{A}}$ is chosen if the final hyperparameter value $\mathbf{A}$ satisfied the condition:

$$a_{\min} \leq \min(\mathbf{A}), \quad \max(\mathbf{A}) \leq b_{\max},$$

where $a_{\min} = -2.5$, $b_{\max} = 10.5$ for the synthetic dataset and $a_{\min} = -5$, $b_{\max} = 11$ for the WISDM and MNIST datasets. We calibrate $\gamma_{\mathbf{A}}$ with a small amount of iterations: $l = 50$ for the synthetic dataset, $l = 10$ for the WISDM and $l = 5$ for the MNIST dataset. Whenever each algorithm shows instability in further experiments (gradient explosion or overflow) we lower the value of $\gamma_{\mathbf{A}}$. The parameter τ_{step} is set to 1 for synthetic dataset and WISDM and $\tau_{\text{step}} = 10$ for the MNIST dataset. The overall results are presented in Table 3. The implementation of the proposed algorithms is available at [2].

Table 3 Experiment results

Algorithm	L, Q	$Q(\theta, \mathbf{A})$	Convergence	E	$E_{0.25}$	$E_{0.5}$
<i>Synthetic</i>						
Random search	(10)	− 171.6	26.2 ± 20.0	1.367	1.410	1.555
Greedy	(10)	− 172.5	30.0 ± 24.5	1.421	1.439	1.536
DrMAD	(10)	− 174.1	40.2 ± 16.1	1.403	1.424	1.512
HOAG	(10)	− 174.7	29.4 ± 24.0	1.432	1.463	1.553
Random search	(12)	− 63.5	32.4 ± 18.7	1.368	1.426	1.546
Greedy	(12)	− 25.5	1.2 ± 0.4	1.161	1.174	1.193
DrMAD	(12)	− 25.1	10.6 ± 0.8	1.157	1.163	1.184
HOAG	(12)	− 25.8	10.8 ± 1.5	1.141	1.149	1.177
<i>WISDM</i>						
Random search	(10)	− 1,086,661.1	22.0 ± 19.3	0.660	0.670	0.690
Greedy	(10)	− 1,086,707.1	15.4 ± 17.2	0.707	0.723	0.769
DrMAD	(10)	− 1,086,708.2	29.2 ± 8.0	0.694	0.708	0.742
HOAG	(10)	− 1,086,733.5	28.2 ± 7.13	0.701	0.724	0.753
Random search	(12)	− 35,420.4	14.4 ± 7.8	0.732	0.755	0.785
Greedy	(12)	− 3552.9	1.0 ± 0.0	0.702	0.730	0.767
DrMAD	(12)	− 26,091.4	50.0 ± 0.0	0.729	0.753	0.816
HOAG	(12)	− 16,566.6	49.0 ± 0.0	0.733	0.755	0.801
<i>MNIST</i>						
Random search	(10)	− 3236.4	7.8 ± 1.9	0.981	0.966	0.866
Greedy	(10)	− 3416.7	10.8 ± 10.4	0.979	0.962	0.860
DrMAD	(10)	− 3469.0	17.0 ± 5.6	0.982	0.962	0.831
HOAG	(10)	− 3748.6	8.6 ± 7.3	0.980	0.961	0.853
Random search	(12)	− 1,304,556.4	14.2 ± 5.7	0.982	0.943	0.814
Greedy	(12)	− 11,136.2	1.0 ± 0.0	0.977	0.952	0.884
DrMAD	(12)	− 1,305,432.9	24.6 ± 0.5	0.982	0.941	0.813
HOAG	(12)	− 280,061.6	24.0 ± 0.0	0.981	0.943	0.819

Bold indicates the best optimization performance

Synthetic dataset The synthetic dataset is generated according to the rule:

$$\mathbf{y} = \mathbf{x} + \boldsymbol{\varepsilon}, \quad \mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

where $m = 40$, $n = 1$.

The regression model $\mathbf{f}$ uses the following features: $\{\mathbf{x}^0, \dots, \mathbf{x}^9, \sin(\mathbf{x}), \cos(\mathbf{x})\}$. The unregularized regression model with such ratio of object number and feature number tends to overfit significantly. The goal of this experiment is to analyze if the hyperparameter optimization could prevent such models from overfitting.

Figure 2 plots the resulting polynomials. Since the model is overparametrized, the optimization without any overfitting control tends to overfit drastically. The real dependency between $\mathbf{y}$ and $\mathbf{X}$ is linear. As we can see, the evidence lower bound criterion (12) gives non-overfitted polynomials, which parameters are close to the linear models.

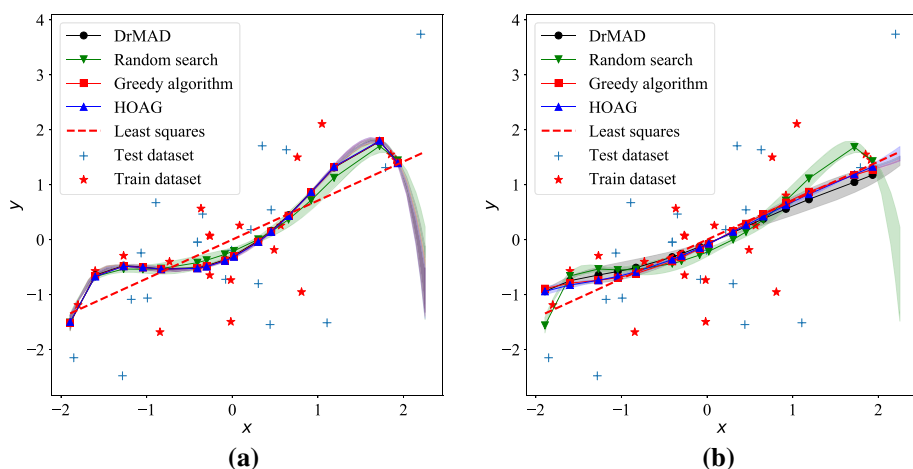


Fig. 2 Resulting models for the synthetic dataset: **a**—cross-validation, **b**—evidence lower bound

WISDM The WISDM dataset contains a set of records from accelerometer. Each record has three coordinates that correspond to the accelerometer axes. As a set of features we use first 199 records from each 200 sequential records. We use l_2 norm of the 200th records as a label and required to predict.

A neural network with 10 neurons on the hidden layer is used:

$$\mathbf{f} = \mathbf{W}_2 \cdot \text{RELU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2,$$

where $\mathbf{W}_1$, $\mathbf{b}_1$ — hidden layer parameters, $\mathbf{W}_2$, $\mathbf{b}_2$ — output layer parameters,

$$\text{RELU}(\mathbf{x}) = \max(\mathbf{0}, \mathbf{x}).$$

Figure 3 shows the plots with RMSE and best validation loss value $\hat{Q}$ for the WISDM dataset. DrMad and HOAG algorithms shows significantly lower results than greedy algorithm. The random search shows good results in case of cross-validation, when the number of hyperparameters is small. When the number of hyperparameters is large the greedy and HOAG algorithm shows good results. HOAG requires more iteration number l than the greedy algorithm for the convergence.

MNIST The MNIST dataset contains a set of images of handwritten digits. We use a neural network with 300 neurons on the hidden layer and softmax output.

Figure 4 plots the error E and quality $\hat{Q}$. For the evidence lower bound criterion, the models with highest $\hat{Q}$ value have also highest error E . Nevertheless, these models show the best results for the stability, when the noise in dataset is high. Figure 5 plots the error E_σ . Models with highest evidence lower bound have the lowest error when the Test dataset is noisy. We interpret this as an ability to select the model with highest generalization ability.

The experiments show that the gradient-based methods give better results than the random search when the number of hyperparameters is large. The best results were obtained by the greedy algorithm. We find that the hyperparameter optimization is unstable when using DrMad algorithm, therefore its learning rate γ_A is set to low values.

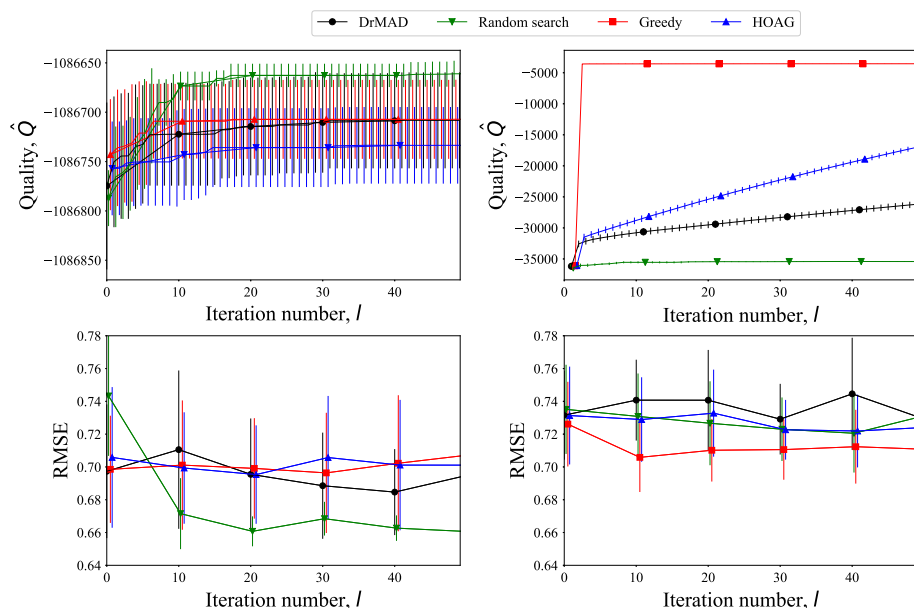


Fig. 3 WISDM, best validation value $\hat{Q}$ and RMSE for cross-validation (left) and evidence lower bound (right)

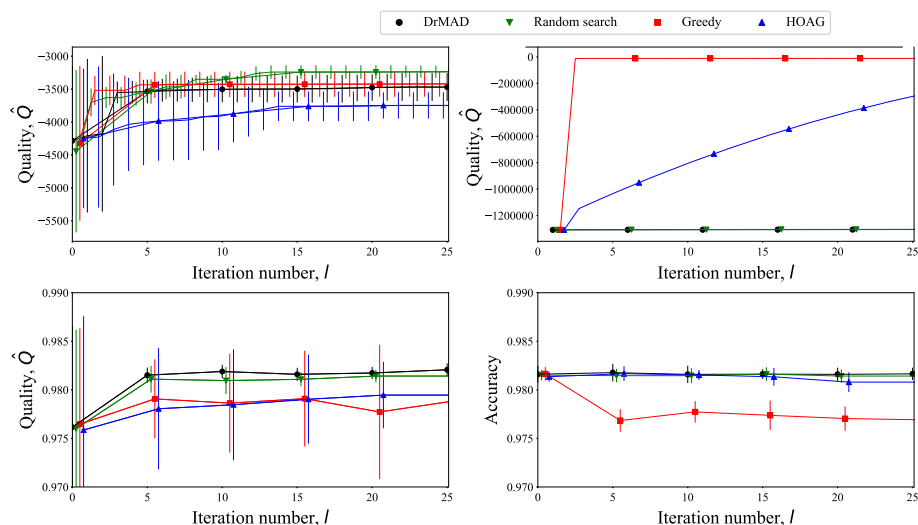


Fig. 4 MNIST, best validation value $\hat{Q}$ and Accuracy for cross-validation (left) and evidence lower bound (right)

5 Discussion

The experiments show that each algorithm perform effectively and therefore the appropriate hyperparameter optimization method should rely on the amount of hyperparameters and the specific of the problem.

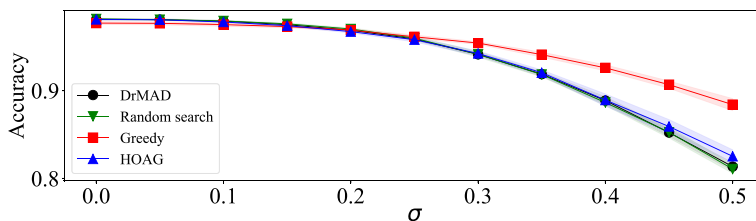


Fig. 5 MNIST, model accuracy with noise in the test dataset. The hyperparameters were optimized with evidence lower bound criterion

When dealing with small amount of hyperparameters random search shows the best results since search procedure can be employed more effectively than gradient-based optimization in low-dimensional hyperparameters space. For the high-dimensional hyperparameter space both HOAG and greedy algorithms show good performance. HOAG algorithm is more preferable if the model optimization problem is expensive. On the other hand we can schedule greedy hyperparameter optimization to make it less expensive as in Luketina et al. (2016).

DrMad algorithm showed rather poor results on the MNIST and WISDM datasets. Perhaps, it is so because of high learning rate γ used in experiments. The large value of the learning rate can make DrMad algorithm instable. Two improvements can be proposed. We can use more stable optimization like Adam or AdaGrad for both parameter and hyperparameter optimization. The second improvement was proposed in Fu et al. (2016): we can use more complicated parameter trajectory approximation to make it more similar to the original parameter trajectory. Opposing to HOAG and greedy algorithms, DrMad optimization has prerequisites for not only hyperparameters optimization but also the metaparameters, i.e., the optimization procedure parameters. The opportunity of such optimization using reversed differentiation was shown in Fu et al. (2016).

The other interesting aspect of our experiments is the relation between the model error (RMSE or Accuracy) and the value of validation loss Q . The models obtained by the evidence lower bound showed higher error rate than the models obtained using cross-validation on the MNIST and WISDM datasets. These models also show greater stability when noise was added to the Test datasets. The evidence lower bound showed significantly better results on the synthetic dataset, when the amount of objects in the Train dataset is small. Therefore we conclude that the evidence lower bound usage is preferable when the model tends to overfit or when the cross-validation usage is too computationally expensive. In Graves (2011) it was noted that the evidence lower bound optimization required more iterations for the convergence. In our experiments we used the same number of iterations both for the cross-validation and evidence lower bound. The more accurate iteration number calibration can improve the final quality of these models.

6 Conclusion and future work

The paper analyzes the gradient-based hyperparameter optimization algorithms. We adapt the analyzed algorithms for general validation functions and evaluated their performance on the MNIST and WISDM datasets. Two model selection criteria are compared: the cross-validation and evidence lower bound.

The experiments show that the gradient-based algorithms are effective when the number of hyperparameters is large. The results show that models obtained using evidence lower

bound have higher error rates than models obtained using cross-validation, but they are also more stable when the test dataset contains a lot of noise.

The authors implement these algorithms as a toolbox available at [2]. The toolbox is developed in Python using Theano (Bergstra et al. 2010) and Numpy (Oliphant 2006) libraries.

In our future work we are planning to develop the analyzed algorithm and to extend gradient-based algorithms to optimize not only hyperparameters, but also the model optimization parameters. The other object of our future research is the difference between the cross-validation and evidence lower bound and the theoretical aspects of their properties for the models with large amount of parameters.

References

- Arlot, S., & Celisse, A. (2010). A survey of cross-validation procedures for model selection. *Statistics Surveys*, 4, 40–79. <https://doi.org/10.1214/09-SS054>.
- Bakhteev, O. (2019). Pyfos: a library for hyperparameter optimization. <https://git.io/fjZks>. Accessed 1 May 2019.
- Bengio, Y., & Grandvalet, Y. (2004). No unbiased estimator of the variance of k-fold cross-validation. *Journal of machine learning research*, 5, 1089–1105.
- Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(Feb), 281–305.
- Bergstra, J., Breuleux, O., Bastien, F., Lamblin, P., Pascanu, R., Desjardins, G., Turian, J., Warde-Farley, D., & Bengio, Y. (2010). Theano: A cpu and gpu math expression compiler. In *Proceedings of the Python for scientific computing conference (SciPy)*, vol. 4.
- Bergstra, J. S., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for hyper-parameter optimization. In: J. Shawe-Taylor, R. S. Zemel, P. L. Bartlett, F. Pereira, & K. Q. Weinberger (Eds.), *Advances in neural information processing systems* (pp. 2546–2554). Granada: Curran Associates, Inc. <http://papers.nips.cc/paper/4443-algorithms-for-hyper-parameteroptimization.pdf>.
- Bishop, C. M. (2006). *Pattern recognition and machine learning (information science and statistics)*. New York: Springer.
- Domke, J. (2012). Generic methods for optimization-based modeling. *AISTATS*, 22, 318–326.
- Farcomeni, A. (2010). Bayesian constrained variable selection. *Statistica Sinica*, 20(3), 1043–1062.
- Fu, J., Luo, H., Feng, J., Low, K.H., & Chua, T.S. (2016). Drmad: distilling reverse-mode automatic differentiation for optimizing hyperparameters of deep neural networks. In *Proceedings of the twenty-fifth international joint conference on artificial intelligence*, pp. 1469–1475.
- Graves, A. (2011). Practical variational inference for neural networks. *Advances in Neural Information Processing Systems*, 24, 2348–2356.
- Grünwald, P. (2005). A tutorial introduction to the minimum description length principle. In P. D. Grünwald, I. J. Myung, & M. A. Pitt (Eds.), *Advances in minimum description length: theory and applications*. Cambridge: MIT Press.
- Hoffman, M. D., Blei, D. M., Wang, C., & Paisley, J. (2013). Stochastic variational inference. *Journal of machine learning research*, 14(1), 1303–1347.
- Hutter, F., Hoos, H.H., & Leyton-Brown, K. (2011). Sequential model-based optimization for general algorithm configuration. In *International conference on learning and intelligent optimization*, pp. 507–523.
- Hwang, S., & Jeong, M. K. (2018). Robust relevance vector machine for classification with variational inference. *Annals of Operations Research*, 263(1–2), 21–43.
- Karaletsos, T., & Rätsch, G. (2015). Automatic relevance determination for deep generative models. *Statistics*, 1050, 26.
- Krstajic, D., Buturovic, L. J., Leahy, D. E., & Thomas, S. (2014). Cross-validation pitfalls when selecting and assessing regression and classification models. *Journal of Cheminformatics*, 6(1), 1–15.
- Kuznetsov, M., Tokmakova, A., & Strijov, V. (2016). Analytic and stochastic methods of structure parameter estimation. *Informatica*, 27(3), 607–624.
- Kwapisz, J. R., Weiss, G. M., & Moore, S. A. (2011). Activity recognition using cell phone accelerometers. *ACM SigKDD Explorations Newsletter*, 12(2), 74–82.
- LeCun, Y. (1998). The mnist database of handwritten digits. <http://yann.lecun.com/exdb/mnist/>. Accessed 1 May 2019.

- Luketina, J., Berglund, M., Greff, K., & Raiko, T. (2016). Scalable gradient-based tuning of continuous regularization hyperparameters. In *International conference on machine learning*, pp. 2952–2960
- MacKay, D. J. C. (2002). *Information theory, inference and learning algorithms*. New York: Cambridge University Press.
- Maclaurin, D., Duvenaud, D., & Adams, R. (2015). Gradient-based hyperparameter optimization through reversible learning. In *Proceedings of the 32nd international conference on machine learning (ICML-15)*, pp. 2113–2122
- Nystrup, P., Boyd, S., Lindström, E., & Madsen, H. (2018). Multi-period portfolio selection with drawdown control. *Annals of Operations Research*. <https://doi.org/10.1007/s10479-018-2947-3>.
- Oliphant, T.E. (2006). A guide to NumPy, vol. 1. Trelgol Publishing USA
- Pedregosa, F. (2016). Hyperparameter optimization with approximate gradient. In *Proceedings of the 33rd international conference on international conference on machine learning*, vol. 48, pp. 737–746
- Salakhutdinov, R., & Hinton, G. (2007). Learning a nonlinear embedding by preserving class neighbourhood structure. *Journal of Machine Learning Research Proceedings Track*, 2, 412–419.
- Salimans, T., Kingma, D. P., & Welling, M. (2015). Markov chain monte carlo and variational inference: Bridging the gap. *ICML*, 37, 1218–1226.
- Snoek, J., Rippel, O., Swersky, K., Kiros, R., Satish, N., Sundaram, N., Patwary, M.M.A., Prabhat, M., & Adams, R.P. (2015). Scalable bayesian optimization using deep neural networks. In: *ICML*, pp. 2171–2180
- Strijov, V.V. (2014). Model genetation and selection for regression and classification problems (dsc thesis)
- Sutskever, I., Vinyals, O., & Le, Q.V. (2014). Sequence to sequence learning with neural networks. In *Advances in neural information processing systems 27: Annual conference on neural information processing systems 2014*, December 8–13 2014, Montreal, Quebec, pp. 3104–3112
- Vishnu, A., Narasimhan, J., Holder, L., Kerbyson, D.J., & Hoisie, A. (2015). Fast and accurate support vector machines on large scale systems. In *2015 IEEE international conference on cluster computing, CLUSTER 2015*, Chicago, IL, September 8–11, 2015, pp. 110–119